SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	K.Prathiba	**Reg. No.:**	192421163
Date:	30/07/2026	**Duration:**	60 minutes

Code of Conduct

I **K.Prathiba** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **K.Prathiba**

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

Introduction

The **AI-Based Personal Finance and Budget Planning System** is designed to help users effectively manage their personal finances using Artificial Intelligence. The system enables users to record income and expenses, create monthly budgets, monitor savings, and receive AI-based financial recommendations. It provides secure financial management and supports informed decision-making through reports and analytics.

System Objectives

- Develop an intelligent financial management system.
- Record and manage income and expenses.
- Generate AI-based budgeting recommendations.
- Help users achieve savings goals.
- Provide financial reports and dashboards.
- Send bill payment reminders.
- Ensure secure storage of financial data.
- Improve financial planning and decision-making.

Functional Requirements

- User Registration and Login
- Profile Management
- Income Management

- Expense Management
- Budget Planning
- Savings Goal Management
- AI Recommendation Engine
- Financial Dashboard
- Report Generation
- Notification System
- Data Backup
- Logout

Non-Functional Requirements

Performance

- Fast system response.
- Quick report generation.
- Handle multiple users.

Security

- User authentication.
- Password encryption.
- Secure database.

Reliability

- Accurate calculations.
- Data consistency.
- System stability.

Availability

- 24×7 access.
- Minimum downtime.

Scalability

- Support increasing users.
- Future AI expansion.

Maintainability

- Easy updates.
- Modular design.
- Easy debugging.

Quality Attributes

- Scalability
- Reliability
- Performance
- Security
- Maintainability
- Availability
- Usability
- Flexibility

2. Select a Suitable Software Architecture

Introduction

Selecting the correct software architecture ensures that the application is scalable, secure, maintainable, and capable of supporting future enhancements.

Selected Architecture

Layered Architecture (MVC + Client-Server)

Architecture Layers

Presentation Layer

- Login Screen
- Dashboard
- Budget Screen
- Reports
- Notifications

Business Logic Layer

- User Management
- Expense Manager
- Budget Manager
- AI Recommendation Module
- Report Generator

Data Access Layer

- Database Access
- Data Validation
- API Integration

Database Layer

- User Database
- Financial Transactions
- Budget Records
- Reports
- AI Models

Why Layered Architecture?

- Easy maintenance
- Better security
- High scalability
- Reusable modules
- Easy debugging
- Supports AI integration
- Faster development
- Suitable for Banking applications

3. Draw the Software Architecture Diagram

Introduction

The software architecture diagram provides a high-level view of the overall structure of the AI-Based Personal Finance and Budget Planning System. It illustrates how different layers and components interact with each other to process user requests, manage financial data, and generate AI-based recommendations. The system follows a **Layered Architecture** consisting of the Presentation Layer, Business Logic Layer, Data Access Layer, and Database Layer. External services such as Bank APIs, Payment Gateway, and Notification Services are also integrated to improve functionality.

Architecture Components

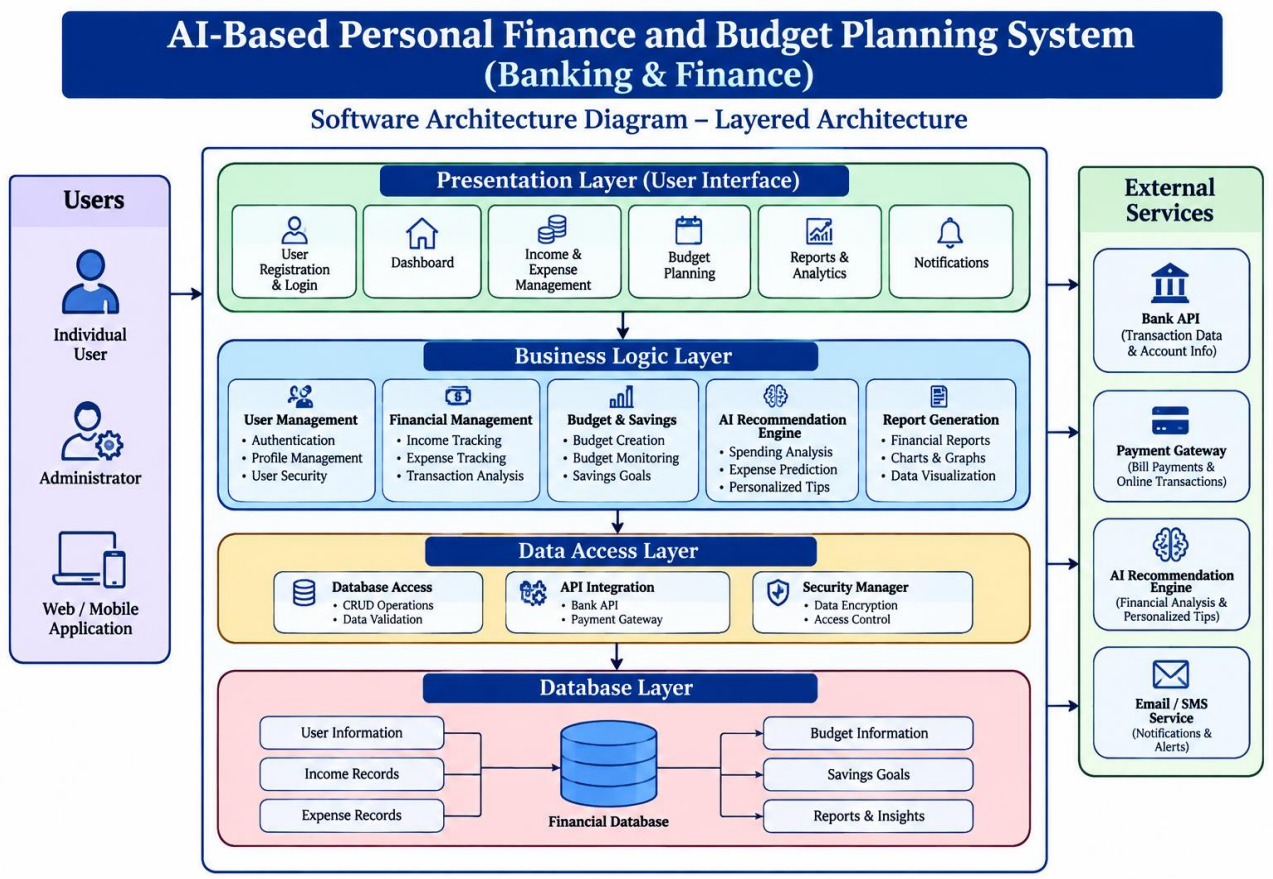
The architecture consists of the following major components:

- User (Web/Mobile Application)
- Presentation Layer (User Interface)
- Business Logic Layer
- Data Access Layer
- Database Layer
- AI Recommendation Engine
- Bank API
- Payment Gateway
- Notification Service (Email/SMS)

Communication Flow

- The user accesses the application through a web or mobile interface.
- The Presentation Layer receives user inputs and sends requests to the Business Logic Layer.

- The Business Logic Layer validates data and processes financial operations.
- The Data Access Layer communicates with the database and external APIs.
- The Database stores user profiles, income, expenses, budgets, and reports.
- The AI Recommendation Engine analyzes financial data and generates personalized suggestions.
- Notification services send alerts for bills, budgets, and savings goals.



4. Explain Components and Their Interactions

Introduction

The AI-Based Personal Finance and Budget Planning System consists of multiple interconnected components. Each component has a specific responsibility and communicates with other components to

provide accurate financial management services. The layered design ensures modularity, maintainability, and secure data processing.

Major Components

1. Presentation Layer

The Presentation Layer acts as the user interface of the application. It allows users to register, log in, manage income and expenses, create budgets, and view reports through an intuitive dashboard.

Responsibilities

- Display user interface.
- Collect user inputs.
- Show dashboards and reports.
- Display AI recommendations.
- Present notifications and alerts.

2. Business Logic Layer

The Business Logic Layer processes all financial operations and business rules. It validates user requests, performs budget calculations, generates reports, and coordinates communication between different modules.

Responsibilities

- User authentication.
- Income management.
- Expense management.
- Budget calculation.

- Savings goal management.
- AI recommendation processing.
- Report generation.

3. Data Access Layer

The Data Access Layer serves as a bridge between the application and the database. It manages data retrieval, storage, updates, and deletion while ensuring secure database access.

Responsibilities

- Database connectivity.
- Execute SQL queries.
- Store financial records.
- Retrieve reports.
- Handle API communication.

4. Database Layer

The Database Layer securely stores all application data.

Stored Data

- User accounts.
- Income records.
- Expense records.
- Budget information.
- Savings goals.
- AI recommendation history.

- Financial reports.
- Notification history.

5. AI Recommendation Engine

The AI engine analyzes financial transactions and generates intelligent budgeting suggestions.

Functions

- Analyze spending patterns.
- Predict future expenses.
- Recommend savings plans.
- Detect unnecessary spending.
- Generate financial insights.

6. External Services

External systems improve application functionality.

- Bank API for transaction synchronization.
- Payment Gateway for bill payments.
- Email/SMS Service for notifications.

Overall Workflow

1. User logs into the system.
2. User enters income and expense details.
3. Business Logic validates the information.
4. Data Access Layer stores data in the database.

5. AI engine analyzes spending patterns.
6. Reports and dashboards are generated.
7. Notifications are sent to users.
8. Users receive personalized financial recommendations.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

Introduction

Quality attributes determine the effectiveness and reliability of the proposed software architecture. The selected Layered Architecture ensures that the system is scalable, secure, maintainable, reliable, and capable of supporting future enhancements.

Scalability

The architecture supports future expansion by allowing new modules to be added without affecting existing components.

Advantages

- Supports increasing users.
- Easy AI enhancement.
- Cloud deployment support.
- Handles large financial datasets.

Security

Financial information is highly sensitive; therefore, multiple security mechanisms are implemented.

Security Features

- User authentication.
- Password encryption.
- Secure API communication.
- Role-based access.
- Database encryption.
- Secure session management.

Performance

The architecture improves system performance through efficient processing and optimized database operations.

Performance Features

- Fast login response.
- Quick report generation.
- Optimized database queries.
- Efficient AI processing.
- Reduced server load.

Reliability

The system provides reliable financial management with accurate calculations and secure data handling.

Reliability Features

- Accurate calculations.
- Error handling.
- Automatic backup.
- Transaction consistency.
- Fault tolerance.

Maintainability

The modular architecture makes future maintenance simple.

Maintainability Features

- Independent modules.
- Easy bug fixing.
- Simple software updates.
- Code reusability.
- Easier testing.

Availability

The application is designed to provide uninterrupted financial services.

Availability Features

- 24×7 accessibility.
- Cloud hosting.
- Backup server.
- Disaster recovery.
- High uptime.

Benefits of Quality Attributes

- Improved customer satisfaction.
- Better software reliability.
- Enhanced security.
- Reduced maintenance cost.
- Faster feature development.
- Long-term system stability.

6.Justify Architectural Decisions

ntroduction

The selected **Layered Architecture (MVC + Client-Server)** is suitable for the AI-Based Personal Finance and Budget Planning System because it provides a structured approach to software development. It separates the application into independent layers, making the system easier to develop, maintain, test, and expand.

Reasons for Selecting Layered Architecture

- Separates presentation, business logic, and data access.
- Simplifies application maintenance.
- Supports secure financial transactions.

- Allows independent module development.
- Improves scalability.
- Enhances code reusability.
- Facilitates testing and debugging.
- Supports future AI integration.

Trade-offs Considered

Although Layered Architecture offers many benefits, certain trade-offs were considered during its selection.

- Increased communication between layers.
- Slightly longer processing time.
- More initial development effort.
- Additional testing for layer integration.
- Higher design complexity than simple architectures.

However, these disadvantages are outweighed by improved maintainability, security, and scalability.

Future Enhancements

The architecture easily supports future system improvements.

- Mobile application integration.
- Voice-based financial assistant.
- Banking API integration.
- Cloud synchronization.
- Investment management module.
- AI fraud detection.

- Multi-language support.
- Predictive financial analytics.
- QR-based payment integration.
- Smart chatbot for financial guidance.

Conclusion

The Layered Architecture provides a robust, secure, and scalable foundation for the AI-Based Personal Finance and Budget Planning System. It effectively supports modular development, AI integration, secure financial data management, and future system enhancements. This architectural approach ensures long-term maintainability while delivering a reliable and high-performance banking and finance application.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	presents a well-organized, professional report.		report needs improvement.	

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25